SISTEMAS OPERACIONAIS II · FAETERJ

Análises Financeiras Brasileiras

Como o paralelismo ajuda no desempenho do modelo

Romulo Ribeiro · Novembro de 2026

O QUE O PROGRAMA SE PROPÕE A FAZER

Dá para prever se uma ação sobe ou cai amanhã?

391
ações da B3 analisadas

11 anos
de 2015 a 2025, 2.729 pregões

438 mil
previsões conferidas com o que aconteceu

1 Ler e limpar

Cotações diárias da B3, só de ações negociadas quase todo dia, com preços corrigidos por eventos da B3 e da CVM.

2 Descrever cada ação

Retorno, volatilidade, indicadores como média móvel e IFR, e quanto ela acompanha o mercado.

3 Prever o dia seguinte

Uma fórmula aprende com os 500 pregões anteriores e diz alta ou queda. Depois, compara com um chute simples.

Trabalho acadêmico, não recomendação de investimento.

ESTRUTURA

Oito arquivos de código, três executáveis

Arquivo	O que tem
comum.h	constantes, structs (Registro, ResultadoAtivo...) e declarações
comum.cpp	leitura, preparação, analisarAtivo, etapasEntreAtivos
relatorio.cpp	saída no terminal e os três CSVs
relatorio_md.cpp	o relatório .md de cada execução
sequencial.cpp	main da versão sequencial
threads.cpp	main e trabalhar com pthread
processos.cpp	main e trabalhar com fork e memória compartilhada
Makefile	gera build/sequencial, build/threads e build/processos
benchmark.sh	roda as três versões 5 vezes e gera o benchmark.md

O ROTEIRO DO MAIN

Os três main seguem os mesmos cinco passos

1 Ler
Os 11 CSVs da B3 (2015 a 2025) e os ajustes de eventos da B3 e da CVM.

2 Preparar
Fica só com ações, ordena por ativo e data e corrige os desdobramentos.

3 Analisar cada ativo
O trabalho pesado, 391 vezes. **É o único passo que muda:** um laço, threads ou processos.

4 Cruzar os ativos
Mercado, beta e correlação. Precisa de todos os ativos prontos.

5 Mostrar e gravar
Resumo no terminal, três CSVs e um relatório .md.

```
// 1. Ler
double inicio = agoraEmSegundos();
vector<Registro> lidos = lerDados();
vector<Ajuste> ajustes = lerAjustes();
tempos.leitura = agoraEmSegundos() - inicio;

// 2. Preparar
DadosPreparados dados = prepararDados(lidos, ajustes);

// 3. Analisar cada ativo (muda por versão)

// 4. Etapas entre ativos
etapasEntreAtivos(dados, geral);

// 5. Mostrar e gravar
imprimirResumo(dados, geral);
gravarArquivos(dados, geral);
```

Cada passo é cronometrado do mesmo jeito: **inicio = agoraEmSegundos()**, faz o trabalho, **tempo = agoraEmSegundos() − inicio**.

COMUM.H

As structs que o programa usa

Registro
Uma linha do CSV: um dia de uma ação.

BlocoAtivo
Onde cada ativo começa e termina dentro de registros.

DadosPreparados
Tudo já lido e limpo. Os trabalhadores só leem.

ResultadoAtivo
O que cada trabalhador produz, um por ativo.
Só números.

ResultadoGeral
Junta os 391 resultados no fim, no pai.

```
struct Registro {
    string data, ativo, empresa;
    double abertura, maxima, minima, fechamento;
    long long volume;
};
struct BlocoAtivo { string ativo; int inicio, fim; };
struct DadosPreparados {
    vector<Registro> registros;
    vector<BlocoAtivo> blocos;
    vector<string> pregoes;
};
struct ResultadoAtivo {  // só números
    double precoMedio, retornoMedio, volatilidade;
    Indicadores ultimo;
    AvaliacaoPrevisao previsao;
    Extremo maioresAltas[10], maioresQuedas[10];
    long long histograma[22];
};
struct ResultadoGeral { vector<ResultadoAtivo> ativos; … };
```

COMUM.H · ESTRUTURAS DE DADOS

Do CSV ao resultado em quatro estruturas

`lerDados()`

`Registro`

Uma linha do CSV: data, ativo, preços, volume, empresa. 2,3 milhões lidas.

Lido uma vez, no início

→

`prepararDados()`

`DadosPreparados`

666.998 cotações de ações e 391 blocos [inicio, fim), um por ativo.

Todos leem, ninguém altera

→

`analisarAtivo()`

`ResultadoAtivo`

Estatísticas, indicadores, previsão, 10 altas e quedas, histograma.

Um por ativo, feito pelos trabalhadores

→

`etapasEntreAtivos()`

`ResultadoGeral`

Todos os ativos, índice de mercado, beta e correlações.

Montado no fim, pelo pai

Por que o ResultadoAtivo é só números?

Porque ele é o único que viaja do filho para o pai, pela memória compartilhada.

1. Depois do fork, cada processo tem a sua memória. O que o filho cria fica só com ele; só o segmento do **shmget** é visto pelos dois.
2. Uma **string** (ou **vector**) não guarda o texto dentro dela, só o endereço de onde ele está, na memória do filho. No pai, esse endereço não leva ao texto.
3. Números e arrays de tamanho fixo ficam **dentro** da struct. Copiou a struct, copiou tudo: o filho grava, o pai lê.

O QUE O PROGRAMA CALCULA

Seis cálculos por ação, três entre ações

Cálculo	O que é	Quando roda
Retorno diário	variação do fechamento, corrigida por desdobramentos	dividido entre N
Estatísticas	preços, retorno médio, volatilidade e volume	dividido entre N
Indicadores	MM20, Bollinger, IFR, MACD, estocástico, ATR, OBV	dividido entre N
Altas, quedas, histograma	10 maiores de cada lado e faixas de 1%	dividido entre N
Previsão	regressão (Ridge) treinada nos 500 pregões anteriores	dividido entre N
Avaliação	acerto contra o chute, com margem de erro de 95%	dividido entre N
Índice de mercado	média dos retornos de todas as ações no dia	no fim, sozinho
Beta	quanto a ação anda quando o mercado anda 1%	no fim, sozinho
Correlação	os pares de ações que mais andam juntos	no fim, sozinho

VERSÃO SEQUENCIAL

Um ativo depois do outro, num fluxo só

```
inicio = agoraEmSegundos();
ResultadoGeral geral;
geral.ativos.resize(numAtivos);                  // uma vaga por ativo
for (int b = 0; b < numAtivos; b++) {            // threads e processos dividem este for
    geral.ativos[b] = analisarAtivo(dados, b);   // o trabalho pesado
}
tempos.porAtivo = agoraEmSegundos() - inicio;
```

Analisar os 391 ativos é **66% do programa**: 10,13 s de 15,43 s.



DIVISÃO DO TRABALHO

Os ativos são intercalados

Exemplo: 3 trabalhadores (T0, T1, T2) e 12 ativos




```
for (int b = id; b < numAtivos; b += total)  // começa no seu id, pula de total em total
```

Como ler o for

id é o número do trabalhador e **total** é quantos são. Com 3, o T1 pega 1, 4, 7, 10...

Ninguém precisa combinar nada: a conta já garante que dois trabalhadores nunca pegam o mesmo ativo.

Por que não em blocos

Os ativos têm de 20 a 2.729 pregões. Em blocos (0–3, 4–7, 8–11), um trabalhador pode pegar vários longos seguidos e terminar depois dos outros.

Intercalando, cada um recebe uma mistura de longos e curtos.

THREADS.CPP

A função de cada thread

Uma memória só, para todas

As threads moram dentro do mesmo processo. Por isso **dados** e **geral**, que são globais, são vistos por todas: nada precisa ser copiado.

O formato que o pthread exige

O pthread_create só aceita funções **void* f(void*)**: recebem um ponteiro e devolvem outro. Para mandar só um número, o id, o código finge que ele é um ponteiro na criação, **(void*)(long)i**, e desfaz isso na primeira linha, **(int)(long)arg**.

Cada uma no seu pedaço

Com 4 threads, a thread 1 faz os ativos 1, 5, 9, 13... e a thread 2 faz 2, 6, 10, 14... Os números nunca se repetem. Todas leem os mesmos dados, mas cada uma só escreve nas vagas dos seus ativos.

```
DadosPreparados dados;   // só LIDO
ResultadoGeral geral;    // escrita só no seu b
int totalThreads;

void* trabalhar(void* arg) {
    int id = (int)(long)arg;
    int numAtivos = dados.blocos.size();
    for (int b = id; b < numAtivos;
         b += totalThreads) {
        geral.ativos[b] = analisarAtivo(dados, b);
    }
    return NULL;
}
```

THREADS.CPP

Criar, trabalhar junto, esperar

1. Reservar as vagas

Um vector guarda os itens num bloco de memória. Quando o bloco enche, o **push_back** cria um bloco maior e muda tudo de lugar; se outra thread estiver escrevendo, escreve no lugar antigo e o resultado se perde. Com **resize**, as 391 vagas já existem antes das threads nascerem: ninguém muda o vetor, só preenche.

2. Criar as outras threads

pthread_create põe a função trabalhar para rodar em paralelo, já com o número da thread. O for começa em 1 porque a 0 é a principal.

3. A principal também trabalha

Em vez de ficar parada esperando, ela faz a parte da thread 0.

4. Esperar todas

pthread_join só volta quando aquela thread termina. Depois do último join, todos os resultados estão prontos.

```
geral.ativos.resize(numAtivos);

vector<pthread_t> ids(totalThreads);
for (int i = 1; i < totalThreads; i++) {
    pthread_create(&ids[i], NULL, trabalhar,
                   (void*)(long)i);
}

trabalhar((void*)0);  // thread 0

for (int i = 1; i < totalThreads; i++) {
    pthread_join(ids[i], NULL);
}
```

PROCESSOS.CPP

Criar a memória compartilhada

1. shmget: pedir a memória ao sistema

Pede ao Linux um espaço para 391 ResultadoAtivo. O que volta é só um número de identificação, como o número de um armário: ainda não dá para guardar nada nele.

2. IPC_PRIVATE e 0600

IPC_PRIVATE: sempre um segmento novo, que outro programa não acha por nome. **0600**: a mesma permissão de arquivos, leitura e escrita só para o dono.

3. shmat: abrir o armário

Pendura o segmento na memória do processo e devolve um ponteiro. A partir daí, **compartilhado[b]** se usa como um vetor comum.

4. Antes do fork, de propósito

Os filhos herdam o segmento já pendurado, no mesmo endereço. O que um filho escreve ali, o pai enxerga.

```
// uma posição de ResultadoAtivo por ativo
int idMemoria = shmget(IPC_PRIVATE,
        numAtivos * sizeof(ResultadoAtivo),
        IPC_CREAT | 0600);
if (idMemoria == -1) {
    perror("shmget");
    exit(1);
}

ResultadoAtivo* compartilhado =
        (ResultadoAtivo*)shmat(idMemoria, NULL, 0);
```

PROCESSOS.CPP

fork: um código, dois processos

O filho nasce com tudo

Recebe uma cópia da memória do pai, com os dados já lidos e preparados. O Linux só copia de verdade o que alguém altera; como os filhos só leem, o fork sai barato.

exit(0): o filho para aqui

O filho nasce no meio do for, na mesma linha que o pai. Sem o exit, ele voltaria para o for, criaria filhos próprios e ainda faria a parte do pai: trabalho repetido.

fflush: esvaziar antes de copiar

O printf não escreve na hora: junta o texto num buffer. O fork copia o buffer junto, e cada filho imprimiria de novo o texto parado lá.

```
fflush(stdout);  // senão o texto sai duplicado
for (int id = 1; id < totalProcessos; id++) {
    int pid = fork();
    if (pid == -1) { perror("fork"); exit(1); }
    if (pid == 0) {
        // FILHO: faz a parte dele e termina
        trabalhar(id, totalProcessos, dados,
                  compartilhado);
        shmdt(compartilhado);
        exit(0);
    }
    // PAI: segue criando o próximo filho
}
trabalhar(0, totalProcessos, dados, compartilhado);
```

PROCESSOS.CPP

Esperar, copiar e apagar

1. wait: o join dos processos

O pai espera cada filho terminar. Além de garantir que os resultados estão prontos, isso libera o filho no sistema; sem wait, ele vira um processo zumbi.

2. Trazer para um vetor normal

O pai copia as 391 posições do segmento para geral.ativos, o vetor que o resto do programa usa.

3. Soltar e apagar

shmdt desprende o segmento do pai, mas não o apaga: o segmento é do sistema, não do processo. **shmctl IPC_RMID** manda o Linux apagá-lo. Sem isso, cada execução deixaria um segmento órfão ocupando memória até alguém rodar ipcrm.

```
for (int i = 1; i < totalProcessos; i++) {
    wait(NULL);
}

ResultadoGeral geral;
geral.ativos.resize(numAtivos);
for (int b = 0; b < numAtivos; b++) {
    geral.ativos[b] = compartilhado[b];
}
shmdt(compartilhado);
shmctl(idMemoria, IPC_RMID, NULL);
tempos.porAtivo = agoraEmSegundos() - inicio;
```

SINCRONIZAÇÃO

Por que não precisa de mutex nem semáforo

Quem mexe em quê	T0	T1	T2
dados (registros, blocos)	lê	lê	lê
ativos[0], [3], [6]...	escreve	—	—
ativos[1], [4], [7]...	—	escreve	—
ativos[2], [5], [8]...	—	—	escreve

Cada posição tem um único dono.

Ler o mesmo dado ao mesmo tempo é seguro. Escrever também, quando ninguém mais escreve no mesmo lugar.

> **Sem condição de corrida: as três versões geram resultados idênticos.**
>
> 391 ações, 50,94% de acerto, as mesmas tabelas.

MAKEFILE

Compilar e rodar

Um executável por versão

Cada um é o arquivo da versão compilado junto com o código comum. Por isso os três fazem exatamente a mesma análise.

Só threads precisa de -pthread

Liga a biblioteca de threads. fork, wait e shmget são chamadas de sistema e já vêm na biblioteca C padrão.

Como rodar

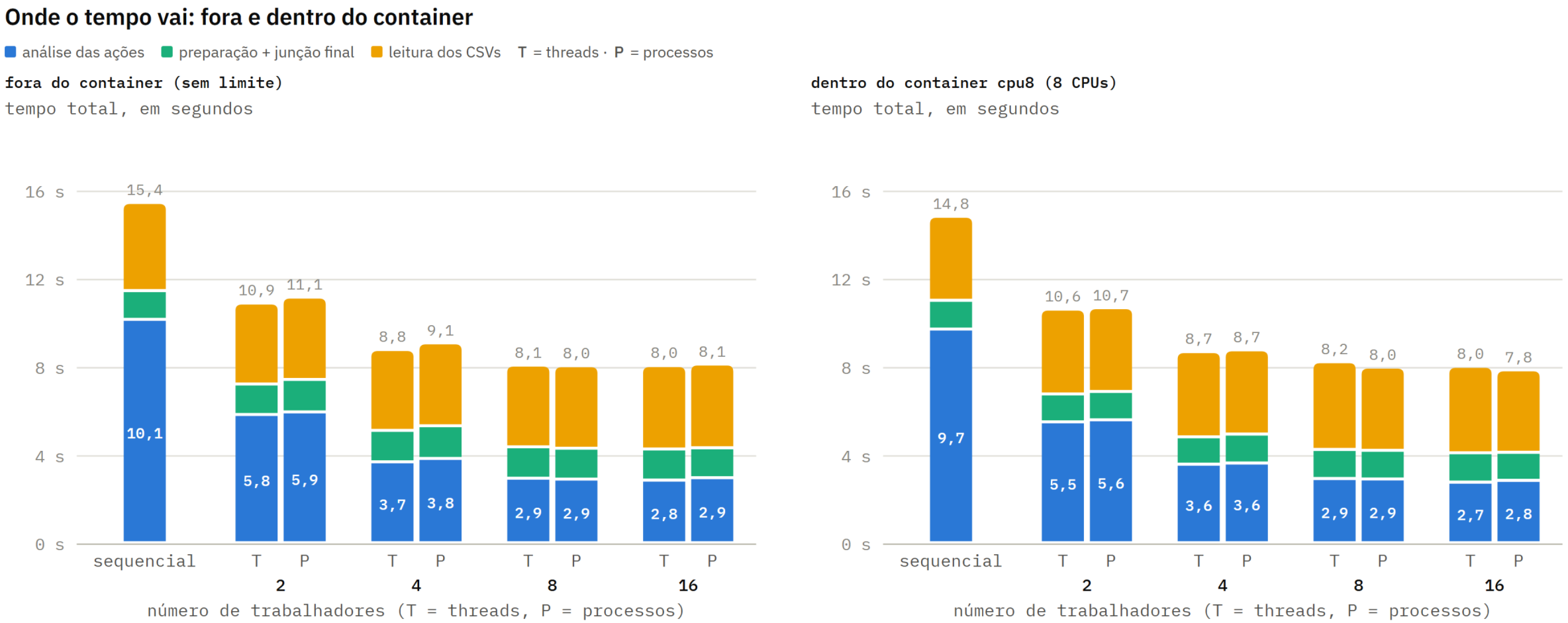
De dentro de analise-mercado/. O número é quantos trabalhadores usar; um código de ativo no fim, como PETR4, detalha só ele. O benchmark.sh roda tudo 5 vezes e gera a tabela de tempos.

```
CXXFLAGS = -std=c++17 -O2 -Wall -Wextra

build/threads: threads.cpp $(COMUM)
    $(CXX) $(CXXFLAGS) -pthread \
      threads.cpp comum.cpp relatorio.cpp \
      relatorio_md.cpp -o build/threads
```

```
$ make -C analise
$ ./analise/build/sequencial
$ ./analise/build/threads 4 PETR4
$ ./analise/build/processos 4
$ bash analise/benchmark.sh 16 5
```

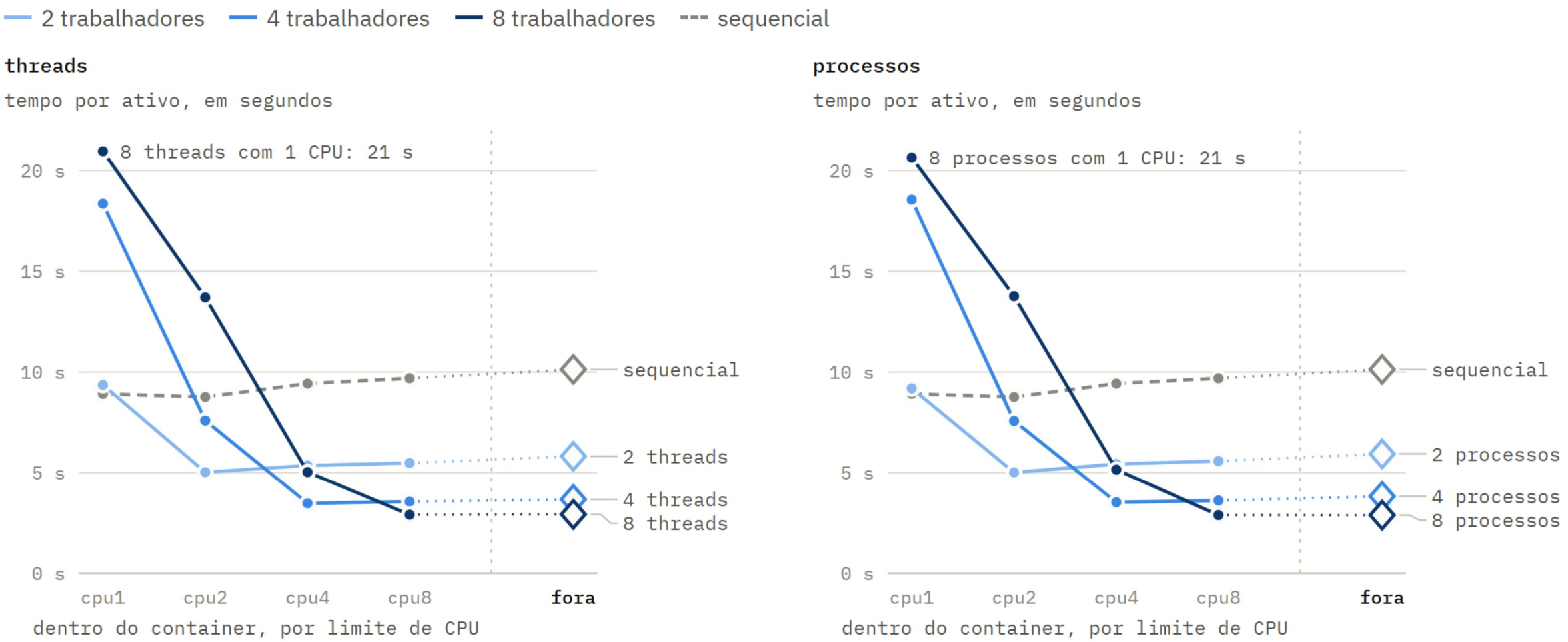
Fora ou dentro do container, o desenho é o mesmo



Com 8 trabalhadores: **8,1 s fora** e **8,2 s dentro**. O container quase não custa nada; o que limita é o número de CPUs.

Mais trabalhadores que CPUs: fica mais lento

Quanto mais CPU o container recebe, mais perto do "fora"



O melhor é um trabalhador por CPU

Tempo por ativo em cada configuração e ambiente

	cpu1	cpu2	cpu4	cpu8	fora
sequencial	8,92 s	8,76 s	9,43 s	9,69 s	10,13 s
threads 2	9,36 s	**5,02 s**	5,35 s	5,48 s	5,82 s
threads 4	18,36 s	7,59 s	**3,48 s**	3,56 s	3,67 s
threads 8	20,96 s	13,71 s	5,02 s	**2,91 s**	**2,92 s**
threads 16	não medido	não medido	não medido	2,75 s	2,84 s
processos 2	9,19 s	**5,01 s**	5,43 s	5,58 s	5,93 s
processos 4	18,56 s	7,57 s	**3,53 s**	3,61 s	3,82 s
processos 8	20,65 s	13,77 s	5,15 s	**2,89 s**	**2,88 s**
processos 16	não medido	não medido	não medido	2,83 s	2,95 s

negrito = número de trabalhadores igual ao número de CPUs do ambiente

Como ler

Tempo da análise das ações, média de 5 execuções. Mais azul, mais rápido.

Negrito

Trabalhadores = CPUs. É o mais rápido de cada coluna, ou quase.

Vermelho

Pior que o sequencial: mais trabalhadores do que CPUs.

Threads ≈ processos

Os dois blocos são quase iguais, linha a linha.

CONCLUSÕES

O que fica

Mesmo resultado	O ativo b sempre vai para a posição b. Quem termina primeiro não muda nada, e nenhuma trava foi necessária.
Threads ou processos	Empatam no tempo. Threads já dividem a memória; processos precisam do shmget e de um resultado feito só de números.
Lei de Amdahl: 2,9×	Só 66% do programa foi dividido. Mesmo com infinitos trabalhadores, o total não fica mais que 2,9× mais rápido; com 8, já chegamos a 1,9×. O próximo gargalo é a leitura dos CSVs.
O modelo já tem pontos fortes	Ele erra menos do que apostar **sempre em alta**. Em algumas ações, como a **EQPA3**, acerta bem mais que o chute. E agora roda **duas vezes mais rápido**: dá para testar melhorias logo.